SWE5001 PRACTICE PROJECT · TEAM 8

# BLOCBUY

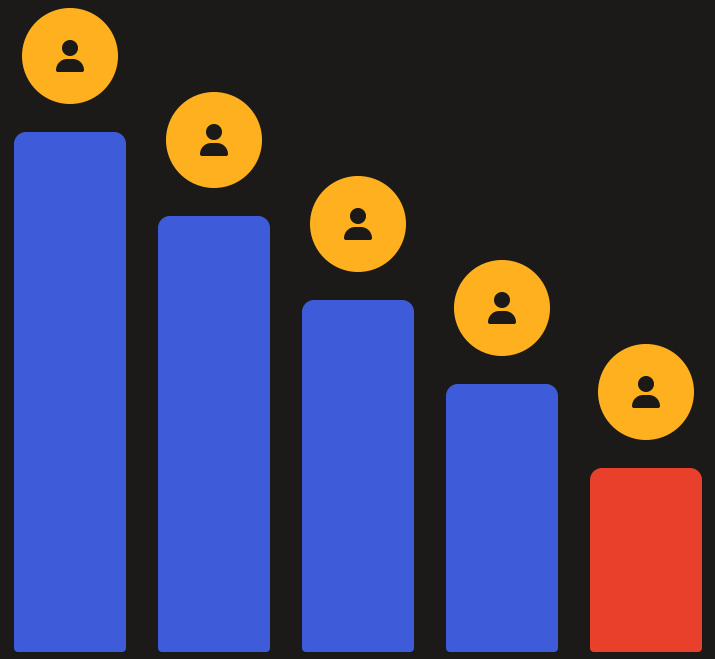
ONE FOR ALL • ALL FOR ONE

## A scalable community group-buying platform

*Three services • event-driven • rebuilt on every release*

Lim Jiechao · Liu Jin · Panpakorn Siripanich · Wang Ziang · Yu Qianshuo

Graduate Certificate in Architecting Scalable Systems · 21 October 2026



*The price drops as each member joins*

# Group buying only works if enough people join in time

## Today: chat groups and spreadsheets

- Buyers can't see deadlines or progress
- Organisers collect money and refund by hand
- Sellers have no place for team prices

## With BlocBuy

- Pay to join a team
- More members, lower price
- Team fills, or everyone is refunded automatically

JC Lim

# A two-sided platform built around one interaction: forming a team

**Seed** a few curated launch offers

**Network effect** each new member lowers the price

## Producers

- Sellers and admins publish offers
- Buyers who start or share a team

CORE INTERACTION

**Form a team at a team price**

## Consumers

- Browse offers
- Join a team
- Track orders

*Consumers become producers when they start or share a team*

**Shared, reusable services**

Identity | Notifications | Payments

**Future seeds:** event tickets · bulk subscriptions · neighbourhood produce boxes

# Scope: the full buying journey, on three services

## Identity service

- Sign in with Google
- Buyer and admin roles

## Commerce service

- Browse team offers
- Start or join a team, and pay
- Team succeeds, or expires with refunds
- Share invites, order history
- Admin listings

## Notification service

- Team succeeded or expired
- Payment received

**Not in scope:** multi-region · real payments (Stripe test mode) · email or push · extracting payment or catalog into their own services (documented evolution path)

# Status: what is verified on AWS, and what is still open

## Verified on AWS · 17 Oct

- Built from nothing: 3 services healthy
  *commerce 2/2 · identity 1/1 · notification 1/1*
- 9/9 smoke checks through CloudFront
- SNS → SQS routing confirmed
  *with synthetic events*
- Full teardown: 115 resources in 709 s
  *absence check clean*
- One task load-tested: 200 req/s
  *0 errors, p95 < 60 ms (16 Oct)*

## Verified on 19 Oct

`[FILL 19 Oct: green tick only with evidence in docs/swe5001/evidence/]`

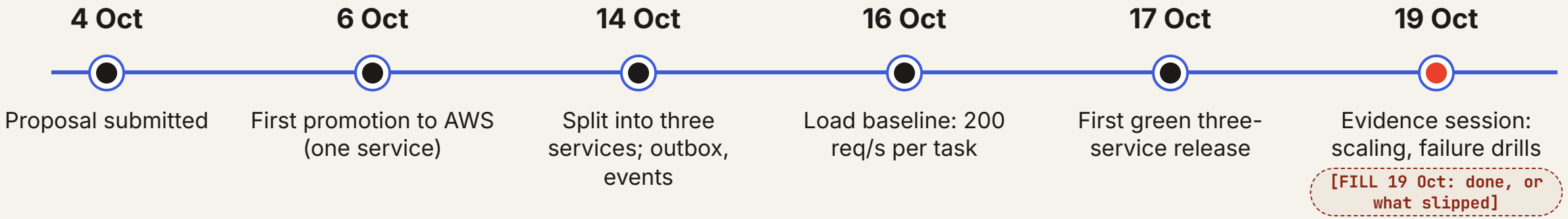
- Auto-scaling under load (V3)
- Queue scaling (V4)
- Self-healing (V5)
- Poison message to DLQ (V7)
- Identity outage drill
- Buyer journey end to end on ECS (#460)

## Still open

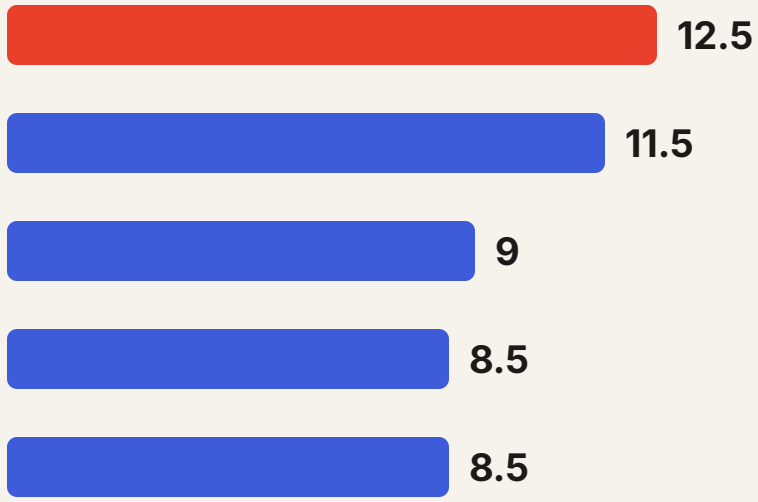
- An interrupted start-up can leave auto-scaling paused
  *manual repair in place*
  `[VERIFY: fix PR open? if not, no "fix in progress"]`
- Database and cache in one zone
  *a cost choice; multi-zone for production*
- Alarm email confirmation (#515)

*Anything not run on 19 Oct moves here with one honest line, e.g. "V4 queue scaling: verified locally (141 s), not yet on AWS".*

# On schedule, about 50 person-days across five people




## Effort by member (person-days) [CONFIRM: by all five, 18 Oct]




*Total ≈ 50 person-days, estimated from the issues each person delivered*

WZ Eric

# Why three services, not five

AD-02 · ADR 0008

## One payment = one database transaction

Payment → Order → Stock → Team

Splitting this needs sagas, so it stays together in **Commerce service**

Identity: used through one small interface

**Identity service**

Notifications: nothing calls them

**Notification service**

Our rule

## Split only where no transaction crosses

✕ **Rejected: five services**
Sagas everywhere, about 50–55 person-days: our whole budget

WZ Eric

# Four decisions shape the logical design

AD-02

## Split by transaction boundary

No distributed transactions to build or debug

AD-04

## Events through an outbox and queues

An event is sent only if the change really committed

AD-05

## One shared login session

Any service checks roles without calling identity

AD-06

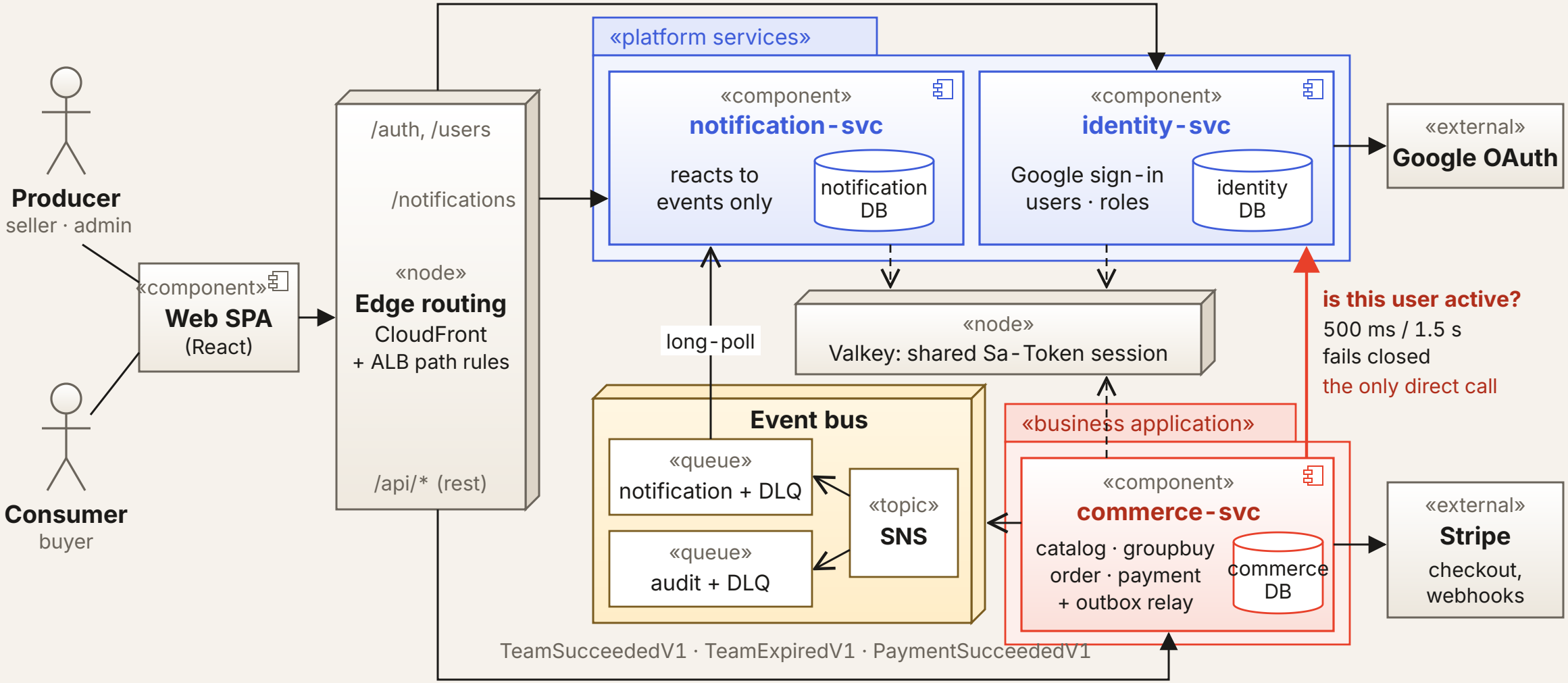
## Fail safe when identity is down

Purchases are refused, never half-done

**Cost of these choices:** notifications lag about 2 s (relay interval); purchases pause during an identity outage

WZ Eric

# Logical view: one direct call, everything else by events




**One direct call** (commerce → identity); everything else by events.

WZ Eric

# Bounded contexts decide the services; events are the contract

## Commerce service

### Catalog
products, team prices

### Group buying
teams, members

### Orders
orders

### Payments
payments, refunds

## Identity service

### Identity
users, roles

## Notification service

### Notification
notifications

## Domain event → integration event

| | |
|---|---|
| TeamSucceeded | TeamSucceededV1 |
| TeamExpired | TeamExpiredV1 |
| PaymentSucceeded | PaymentSucceededV1 |

- Inside a service: free to change
- Between services: versioned, IDs only

[CONFIRM: Option A (this) or B: context map, Eric]

WZ Eric

# How producers and consumers use the platform, and what reuse costs

## Producers

- Sellers and admins publish listings
- Buyers start and share teams

**Identity**
one account for both sides

**Commerce**
offers, teams, payments

**Events**
any new consumer just subscribes

**Notification**
tells people what happened

## Consumers

- Browse and join
- Track orders
- Get notified

## Measured reuse

`[FILL 19 Oct: from PR; drop row 1 if not found by Tue]`

| Added to the platform | What it took | Commerce changes |
|---|---|---|
| **Audit consumer** | `[FILL 19 Oct: N]` lines in `[FILL 19 Oct: M]` files, PR #`[FILL 19 Oct: …]` | 0 |
| **A new service** | One entry in the Terraform services map (`ecs-service` golden-path module) | 0 |

### One public contract

`contracts/openapi.json` → generated TypeScript client
Events versioned (V1), IDs only

**Every new service starts with**

Login & roles · Error format · Tracing · Outbox · Duplicate protection

`blocbuy-platform-starter`

LJ Liu Jin

# Physical decisions: managed, disposable, low cost

## Fargate containers on ARM

No servers or cluster to run · ~20% cheaper · EKS control plane alone ≈ US$73/month

AD-07 · 13

## Disposable environment

Built per session or release, then destroyed

AD-12

## One database server

A database per service: isolation at low cost; own instance is the scale-out path

AD-11

## Small NAT instance

≈ US$4 instead of US$43 a month

AD-10

## No stored cloud keys

Short-lived pipeline tokens · secrets in Parameter Store

AD-14

MEASURED, NOT ESTIMATED

**US$0.49**

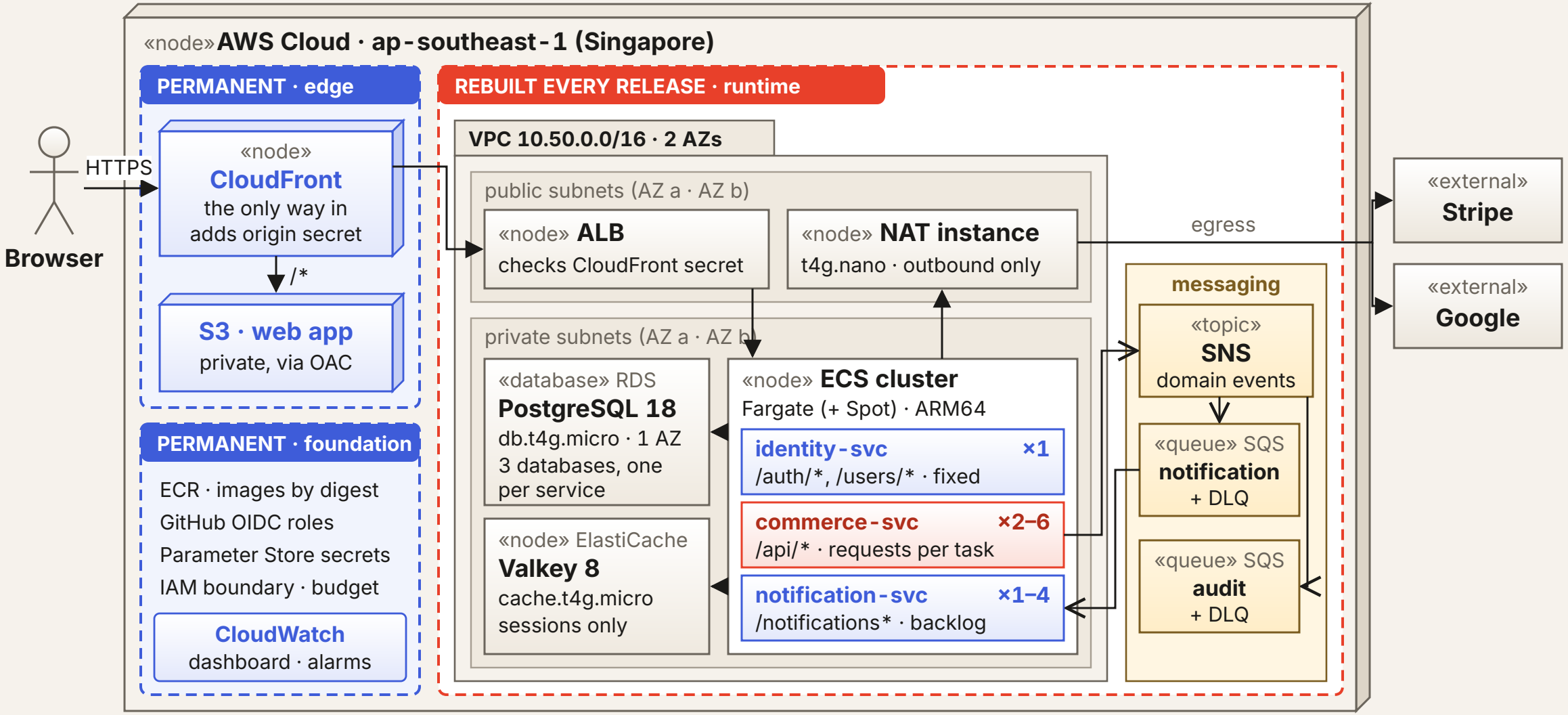
in total for the runtime's database, cache and containers across **7 up/down sessions**, 15–17 Oct

AWS Cost Explorer, filtered to the runtime services

**Trade-off:** less redundancy (one NAT, single-zone database), acceptable for this project

LJ Liu Jin

# Physical view: private services behind CloudFront, across two zones



LJ Liu Jin

# Each service scales on its own signal, and how far it goes

**Commerce**
ALB requests per task (target 12,000/min) → **2 to 6 tasks**

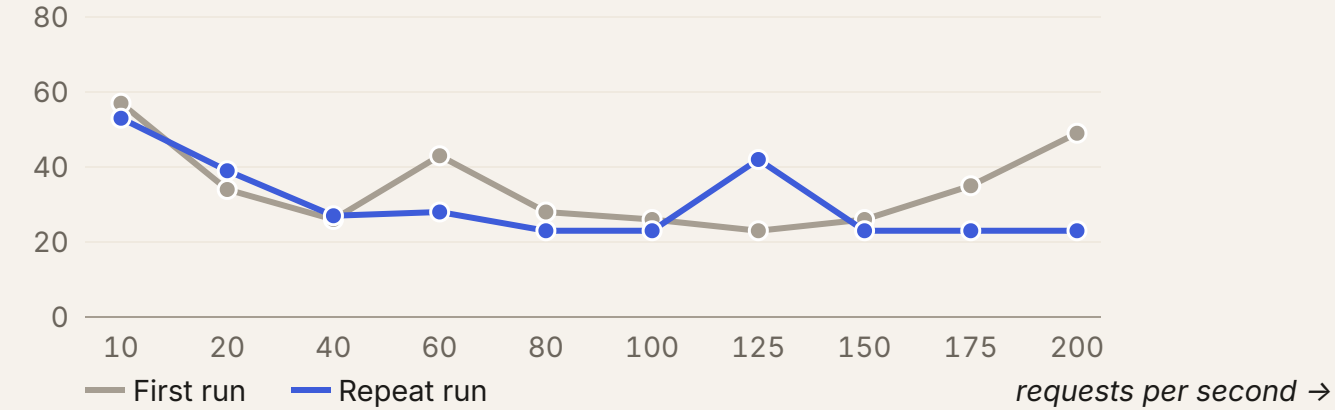
**Notification**
SQS backlog per task (target 50) → **1 to 4 tasks**

**Identity**
Fixed at **1 task**

## Capacity statement

**Measured:** `[FILL 19 Oct: req/s, three services]`
**Projected** ceiling at max tasks: ≈ 6 × 200 = **1,200 req/s** for browse **(projection, not measured)**
**Next bottleneck:** the single database and the single outbox relay
`[VERIFY: ~50 events/s measured or estimated?]`

**Load test, one task, 16 Oct: response time (p95, ms)**




| **200** req/s | **< 60** ms | **0** errors |
|---|---|---|
| highest tested, not saturated | 95% of responses | 57,600 requests |

**Three-service shape, 19 Oct (V3):**
`[FILL 19 Oct: N tasks served X req/s at p95 Y ms, Z% errors]`
Place the CloudWatch crop here: “Services: task count” + “Edge: ALB latency” over the V3 window. If V3 did not run, delete this box.

LJ Liu Jin

# Each service owns its data




**Each service reads only its own tables**

per-service roles; enforced by grants and tested

**The cache holds sessions only**

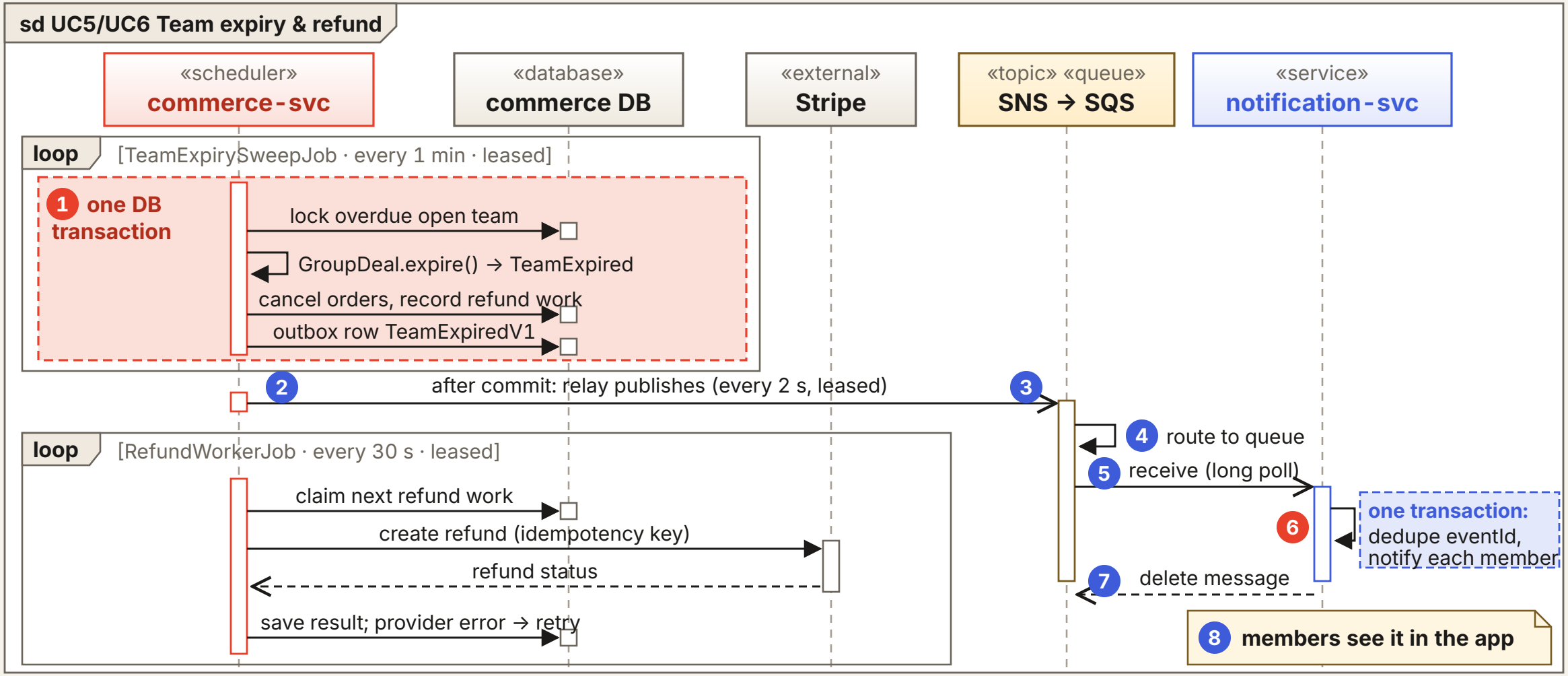
**No card data stored; events carry IDs only**

**Concurrency**

Row locks in a fixed order (**team → intent → payment**) settle the last-seat race. Scheduled jobs hold a database lease, so one task runs them however many are up.

YQ Yorik

# Team expiry, end to end: one transaction in, one transaction out




[VERIFY: step 8: each member or team owner?]

[CONFIRM: swap go/no-go after Yorik rehearses; fallback B15]

YQ Yorik

# Failures are expected, and handled



If...

## A message arrives twice

Ignored: its (event ID, consumer) is already recorded



If...

## A message keeps failing

Moved to a dead-letter queue after 5 tries; an alarm fires



If...

## Publishing fails

The event stays in the outbox and is retried



**No lost events**

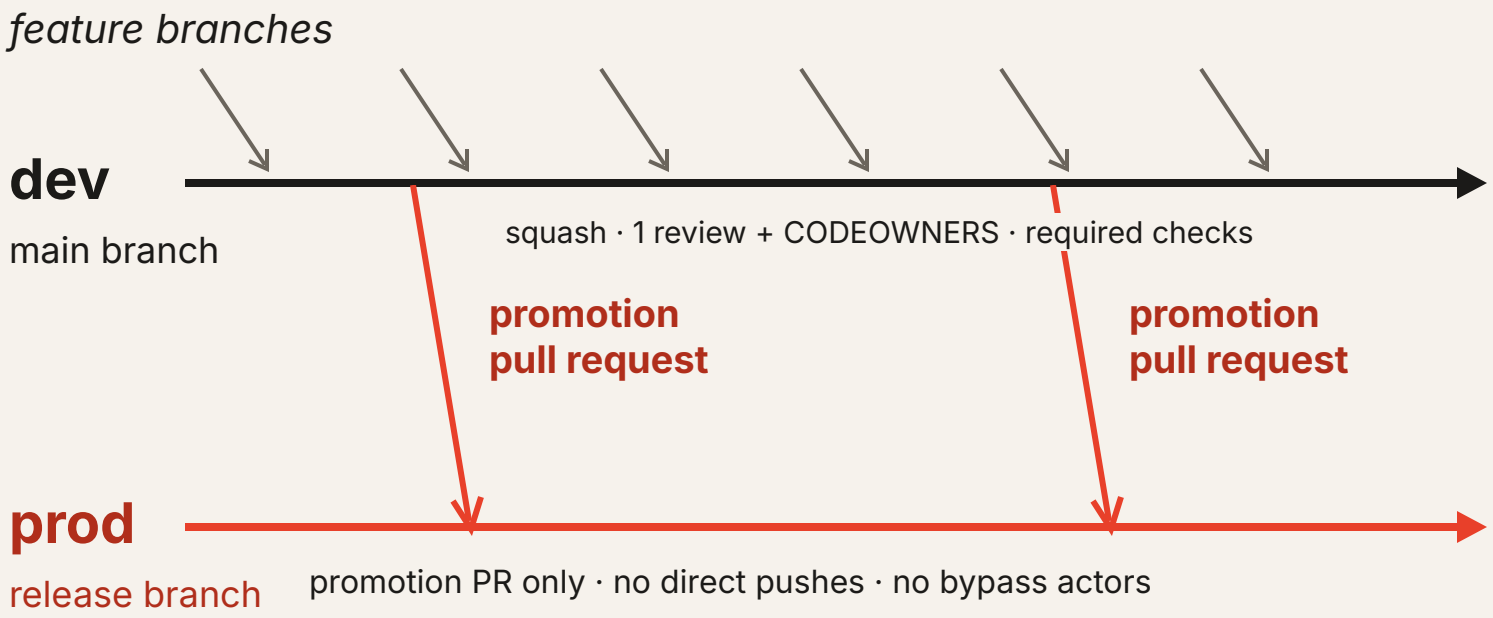


**No double notifications**



**Bad messages isolated**

# One main branch; production changes only by promotion




Review + automated checks

No direct pushes to prod

## Artefacts

**Container images**
one ARM64 image per service, stored by digest, not tag

**Release record**
which digests are live

**API contract**
one file in the repository

# Every pull request is checked before it can merge

## Build & test

- Unit tests and API contract check
- ≥ 80% coverage on new code (SonarCloud)

## Container images

- Build every service
- Boot each one [VERIFY]

## Security

- Secret scanning (gitleaks)
- Dependency and image scanning (Trivy)
- Static analysis [VERIFY: CodeQL runs?]

## Infrastructure

- Terraform fmt, validate, test
- Policy checks (checkov)
- Plan posted to the PR

**5,252** workflow runs, 16 Sep – 16 Oct

**94% / 95%** backend / frontend green

**8.6 / 3.1 min** median run, backend / frontend

**If any check fails, the pull request cannot merge**

# One promotion builds, deploys, tests and releases, then destroys it all

**Approve**
promotion pull request

**Build**
container images
[VERIFY: run log]
first run 6 Oct: 2.5 min

**Create + deploy**
environment and smoke tests
≈ 32 min
first run 6 Oct: 13 min

**Release**
mark images as live
[VERIFY: run log]
first run 6 Oct: 1 min

**Destroy**
and check nothing is left
≈ 12 min
first run 6 Oct: 10 min

**51 min** end to end, three services, 17 Oct (#534)

28 min on 6 Oct with one service. Includes queueing and approval gaps between jobs [VERIFY]. 115 resources destroyed in 709 s.

**Destroy always runs**
even if an earlier step failed

**Failed deploy rolls back**
the ECS circuit breaker restores the last good version

Release digests in Parameter Store are the last-known-good set

Every release is a rebuild from zero, so every release also tests disaster recovery

# What the real cloud taught us

1

## Mocks passed, the real deploy failed

Missing cloud permissions only showed up in a real run (#527)

**Test permissions for real**

2

## Events lost right after deploy

New SNS subscriptions take up to 15 minutes to route; nothing errors, messages just go nowhere

**Deploy now waits for routing**

3

## Auto-scaling silently paused

A retried start-up reported success but left scaling suspended

**Check scaling, don't assume it**

# No standing keys: who and what can touch AWS

**People**
IAM Identity Center

admins (3) · developers (2)
1-hour sessions

**Machines**
GitHub OIDC

separate plan / deploy / apply roles
**no AWS keys in GitHub**

**Disclosed exception**
2 IAM users for SSM port forwarding (1 active key), bounded by the SCPs

**AWS Organization** management account runs no workloads

**Root** SCP · Org-Baseline SCP · Region-Pin: ap-southeast-1 + global

**Sandbox OU** SCP · Sandbox-Guardrails
no expensive services · small instance sizes only

**team8 project account**

**SWE5006 · dev**
EC2 host

**SWE5001 · prod**
ECS runtime, rebuilt per release

**Guardrails above the account:** SCPs bind even the account's admins and root user

# Live demo

**1** **Buy as a customer, across three services**

Functionality

**2** **Auto-scaling under a load test**

Scalability, performance

**3** **Stop a server, nothing breaks**

Availability

**4** **Identity goes down, payments stay safe**

Consistency, security

**5** **Event backlog and the dead-letter queue**

Scalability, reliability

**6** **The pipeline that deployed it**

Maintainability

- **Load test running since 16:28 · everything you see is live on AWS** `[CONFIRM: else "Live on AWS; recorded steps are marked"]`

# Thank you

## Questions?

**WHAT WE HOPE YOU REMEMBER**

- Split where no transaction crosses
- Events never lost or doubled
- Fails safe when identity is down
- Rebuilt from scratch every release

`[FILL 19 Oct: N tasks at X req/s]`
p95 `[FILL 19 Oct: Y]` ms (V3)

**US$0.49**
for 7 full environment sessions

**51 min**
approved PR → released, tested, destroyed

github.com/PPsyrius/iss-groupbuy-team8 · d3lvq02o2e7tcn.cloudfront.net

*If V3 failed: replace the first number with "200 req/s per task, 0 errors (one task, 16 Oct)".*

# Backup slides: index


B1a **Architecture decisions AD-01 to AD-06 (logical)**
Why X over Y (logical)

B1b **Architecture decisions AD-07 to AD-14 (physical)**
Why Fargate, NAT instance, one RDS, accounts

B2 **Known limitations AISS-01 to AISS-06**
Single points of failure, coarse commerce, /internal

B3 **Context map**
What DDD actually changes

B4 **Aggregates and state machines**
GroupDeal / Order lifecycle

B5 **Layers and enforced boundaries**
How boundaries are kept from eroding

B6 **UC4: join a team by paying**
The payment path in detail

B7 **Technology and services**
Exact services, sizes, versions

B8 **Full findings list**
What else went wrong

B9 **Infrastructure as code and actual bill**
Stacks, lifetimes, real cost

B10 **API access for producers and consumers**
Platform Engineering: robust API access

B11 **Analytics: what the platform measures**
Analytics success criterion

B12 **Scale-out and resilience results**
Evidence behind slides 5, 14, 17

B13 **Logical view, simplified**
Plainer companion to slide 9

B14 **Physical view, simplified**
Plainer companion to slide 13

B15 **Team expiry, numbered steps**
Plainer companion / fallback for slide 16


*Click a row (or type its number in the URL as #B6). Slide 24 stays up during Q&A; open a backup only when a question needs it.*

WZ Eric

# Architecture decisions AD-01 to AD-06 (logical)

| ID | Decision | Trade-off / rationale |
|---|---|---|
| AD-01 | Strangler, not big bang: one codebase, two topologies (monolith on dev, distributed on ECS) | Commerce code churned in 46 commits in Sep → split by assembly, not relocation (ADR 0008) |
| AD-02 | Split along transactional consistency boundaries, not nouns | Settlement and expiry stay single-transaction → no sagas. 5-service split rejected (50–55 person-days) |
| AD-03 | Database per service, role per service | No cross-context joins; enforced by grants + ArchUnit |
| AD-04 | Transactional outbox → SNS → SQS + DLQ, idempotent consumers (ADR 0007) | At-least-once + dedupe = effectively-once; no broker to operate |
| AD-05 | Shared Sa-Token session in Valkey, roles in session | No per-request call to identity for authorisation |
| AD-06 | One sync call commerce → identity, fail closed | Outage → 500 → Stripe redelivers; never wrongly rejects a buyer |

LJ Liu Jin

# Architecture decisions AD-07 to AD-14 (physical)

| ID | Decision | Rationale |
|---|---|---|
| **AD-07** | ECS Fargate ARM64, not EKS | EKS control plane ≈ US$73/mo, more than all Fargate compute; ARM ~20% cheaper than x86 |
| **AD-08** | Fargate base + Spot (1 : 3) above the minimum | Cheap burst capacity, baseline protected |
| **AD-09** | CloudFront → ALB with origin-secret header; ALB default 403 | TLS at the edge; ALB unusable except through CloudFront |
| **AD-10** | fck-nat t4g.nano instead of NAT Gateway | ≈ US$4 vs ≈ US$43 per month |
| **AD-11** | One RDS instance, three databases | Logical isolation at demo cost; own instance is the scale-out path, no code change |
| **AD-12** | Persistent foundation/edge, disposable runtime | ≈ US$0.12–0.20/h while up vs ≈ US$125/mo 24×7 (proposal estimate) |
| **AD-13** | No K8s / Argo CD | 8–12 days of setup before any feature work (study 03) |
| **AD-14** | Dedicated member account in an AWS Organization; Identity Center for people, OIDC for CI, SCP guardrails | Isolated blast radius and billing; short-lived credentials; limits the account admin can't lift |

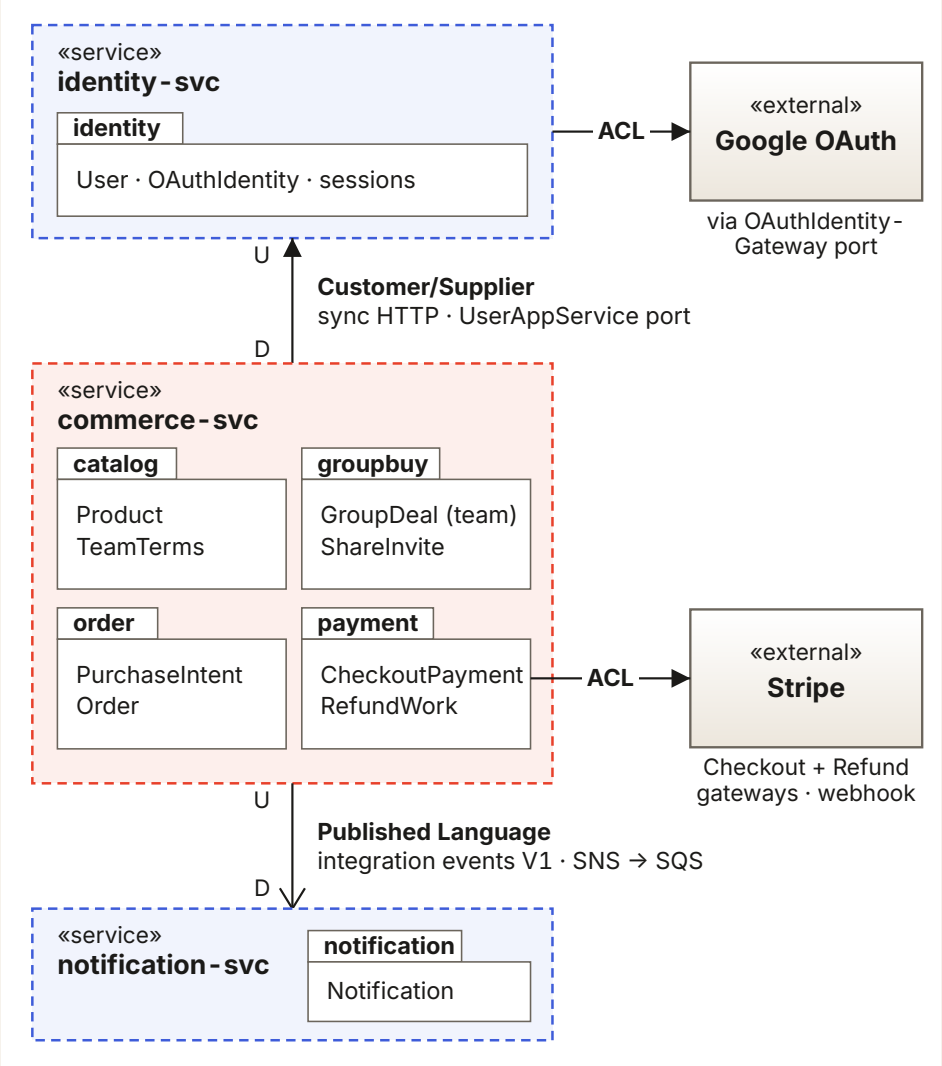
LJ Liu Jin

# Known limitations AISS-01 to AISS-06

| ID | Limitation | Resolution path |
|---|---|---|
| AISS-01 | RDS single-AZ, single Valkey node, one fck-nat | Cost-driven; Multi-AZ, replicas, NAT per AZ |
| AISS-02 | commerce-svc is coarse | Extract payment next, with a saga for fulfilment/refund |
| AISS-03 | Identity outage fails checkout closed | By design; Stripe redelivery; cache or circuit breaker later |
| AISS-04 | /internal/** relies on network isolation only | Add a service token or mTLS |
| AISS-05 | One shared RDS instance | Move a DB to its own instance, no code change |
| AISS-06 | Distributed cold start ≈ 32 min (17 Oct, ×2), 15 min of it the SNS filter wait | Pre-warm on the day; edge stays persistent |

WZ Eric

# Context map




- **identity → Google:** anti-corruption layer via the OAuthIdentityGateway port
- **identity → commerce:** Customer/Supplier, sync HTTP through the UserAppService port (the one direct call)
- **commerce → Stripe:** anti-corruption layer (Checkout and Refund gateways, webhook)
- **commerce → notification:** Published Language, integration events V1 over SNS → SQS

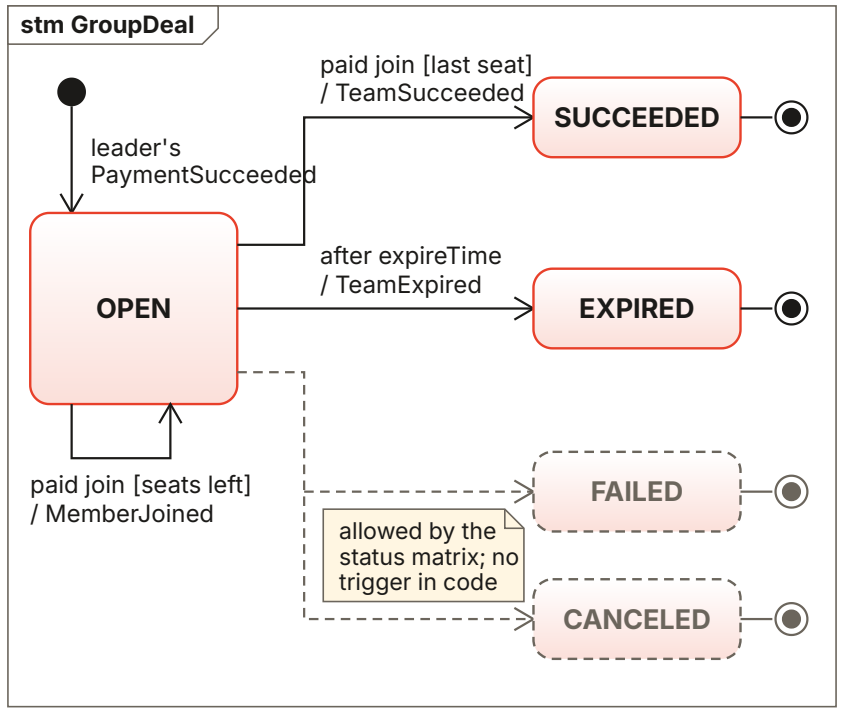
Four contexts share one service because they share transactions; the map shows where the network boundaries are allowed to be.

WZ Eric

# Aggregates and state machines

- **GroupDeal (team):** only OPEN may move, once
- **Order:** PENDING → CONFIRMED | CANCELLED
- **PurchaseIntent** is the order-side aggregate (ADR 0001)
- Also: CheckoutPayment, RefundWork, Product + TeamTerms, Notification
- Domain events: MemberJoined, TeamSucceeded, TeamExpired, PaymentSucceeded
- In-process subscribers run before commit (ADR 0002), so in the same transaction the outbox writer maps them to versioned integration events

**Be ready to explain:** FAILED and CANCELED are allowed by the status matrix but have no trigger in code.



WZ Eric

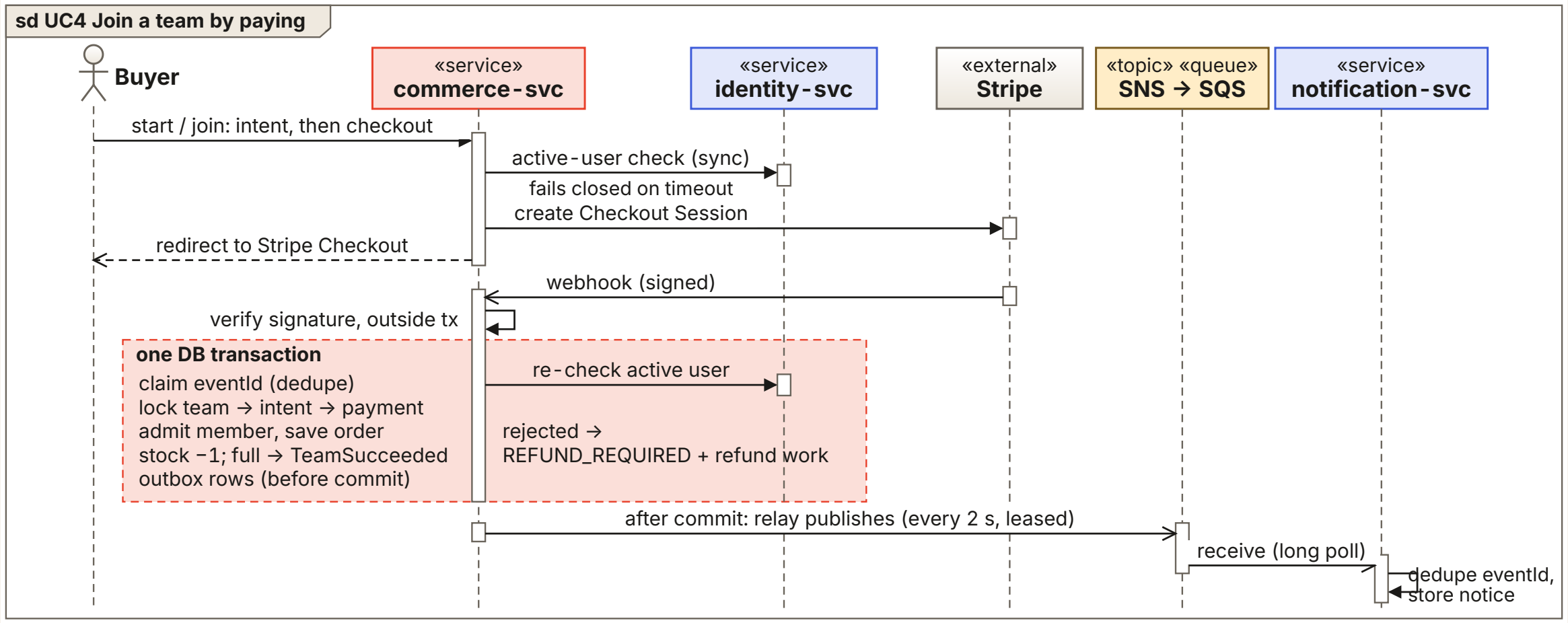
# Layers and enforced boundaries

| Layer | Module | Contains |
|---|---|---|
| Client | `blocbuy-client` | DTOs, integration event contracts (TeamSucceededV1, TeamExpiredV1, PaymentSucceededV1) |
| Adapter | `blocbuy-adapter` | REST controllers, Result envelope, GlobalExceptionHandler |
| Application | `blocbuy-application` | Commands, queries, app services, outbox relay |
| Domain | `blocbuy-domain` | Aggregates, domain events, ports (framework-free) |
| Infrastructure | `blocbuy-infrastructure` | MyBatis, Stripe, Redis, SNS/SQS transports |
| Assemblies | `services/*-svc` | Thin Spring Boot entry points that select contexts |

**Boundaries are tested:** 18 ArchUnit rules, e.g. `commerce_reaches_identity_only_through_user_port`, `nothing_depends_on_notification`

YQ Yorik

# UC4: join a team by paying




Signature verified outside the transaction; one transaction for dedupe, lock order, admit, order, stock and outbox; the active user is re-checked inside it.

LJ Liu Jin

# Technology and services

| Concern | Choice |
| --- | --- |
| Edge | CloudFront (persistent: OAuth redirect and Stripe webhook depend on its domain), S3 SPA bucket |
| Compute | ECS Fargate ARM64, 0.5 vCPU / 1 GB, circuit breaker + rollback, Service Connect |
| Scaling | commerce: ALB requests/target, min 2 / max 6, 60 s out / 180 s in · notification: SQS backlog per task (target 50), 1–4 · identity: fixed 1 |
| Data | RDS PostgreSQL 18 db.t4g.micro (TLS forced) · ElastiCache Valkey 8 cache.t4g.micro (TLS + AUTH) |
| Messaging | SNS (KMS) → SQS (visibility 60 s) + DLQ (maxReceive 5), filter policy |
| Secrets | SSM Parameter Store SecureString; DB passwords via write-only arguments (never in state) |
| Observability | CloudWatch dashboard (Edge · Services · Messaging), DLQ > 0 and queue-age alarms, X-Trace-Id |
| App stack | Java 21, Spring Boot 4, MyBatis, Sa-Token, Stripe · React 19, TanStack, HeroUI v3 |

PS Pan

# Full findings list

| Finding | Fix / status |
|---|---|
| First promotion: 5 failed runs (digests, tfvars, SG tag deny, early 503) | #405–#417; green 6 Oct |
| Fargate read-only root FS crashed Tomcat on /tmp | VOLUME ["/tmp"] |
| Tagging API returns already-deleted ARNs | Per-type describe checks in env-status |
| Flyway default privileges exposed tables to all roles | afterMigrate grants + ServiceGrantsIT |
| Outbox relay marked unsent rows failed on a mid-batch error | Caught in review (#485) |
| Consumer flag never set on ECS → events unconsumed | #512 |
| RegisterScalableTarget AccessDenied: default tags also need TagResource | #527, scoped to Course=swe5001; in effect 17 Oct |
| Fail-closed messaging gate stopped up twice: alarm email unconfirmed (#520, #528) | #529 testing opt-out; green 17 Oct. Confirm emails, opt-out off (#515) `[FILL 19 Oct]` |
| Interrupted bootstrap restores suspended scaling | Manual repair only `[FILL 19 Oct: fix status]` |
| SCP denied CloudFront metrics (us-east-1) | #530: SCPs consolidated, scoped exception |

PS Pan

# Infrastructure as code and actual bill

| Stack | Lifetime | Contents |
|---|---|---|
| foundation | persistent | state bucket, GitHub OIDC roles + permissions boundary, ECR, SSM |
| edge | persistent | CloudFront + S3 SPA (origin switched to the new ALB on each up) |
| runtime | disposable | VPC, ECS, ALB, RDS, Valkey, SNS/SQS, dashboard, alarms: 113–115 resources |

- `env-up.sh / env-down.sh / env-status.sh` (absence check with per-type describe calls)
- Up ≈ 13–15 min (monolith), ≈ 32 min (distributed, 17 Oct ×2); down ≈ 9–13 min (115 resources in 709 s on 17 Oct)
- Golden-path `ecs-service` module: a new service is one map entry
- Apply-role guardrails: IAM only under `/swe5001/`, tag- and name-conditioned denies

**Actual AWS bill** (whole account, Cost Explorer): Sep US$78.51; 1–17 Oct ≈ US$65 (≈ US$4/day), mostly the always-on SWE5006 EC2 host + ALB. The SWE5001 runtime's RDS + Valkey + ECS: **US$0.49** across 7 up/down sessions (15–17 Oct).

WZ Eric

# API access for producers and consumers

**Producers & consumers**
web SPA, future partner clients

→

**CloudFront**
the only way in; adds origin secret

→

**ALB path rules**
/auth, /users, /notifications, /api/*; default 403

→

**Services**
identity · commerce · notification

## Built

- Public contract `contracts/openapi.json`, committed; TypeScript client generated, not committed; contract check on every PR
- Integration events versioned (...V1); consumers ignore unknown versions; producers can add V2 without a lock-step release
- `/internal/**` not routed by the ALB, peer security groups only, absent from the public contract (InternalPathsContractTest)

## Not built (be upfront)

- API gateway, per-client keys, rate limiting
- Service-to-service token or mTLS (AISS-04)
- **Path:** API Gateway, or ALB + WAF rate rules behind the CloudFront origin; a service token on `/internal/**`

JC Lim

# Analytics: what the platform measures

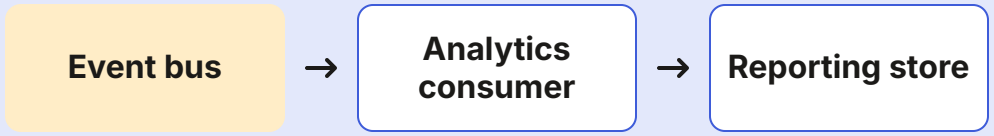
## Operational, in use

- CloudWatch dashboard `blocbuy-swe5001`: edge, services, messaging
- V2 load results set the 12,000 req/target/min scaling target
- DLQ and queue-age alarms

## Business, in the data, not built

- Team fill rate and time-to-fill per product
- Referral attribution from share invites (UC7)
  `[VERIFY: data in biz_* tables?]`
- Conversion from team start to success; refund rate by product

**NEXT STEP**




`[FILL 19 Oct: dashboard crop]`
Crop of dashboard **blocbuy-swe5001**, all three rows, from the 19 Oct session.

A read-only consumer on the same bus, the same pattern as audit: no change to commerce.

JC Lim

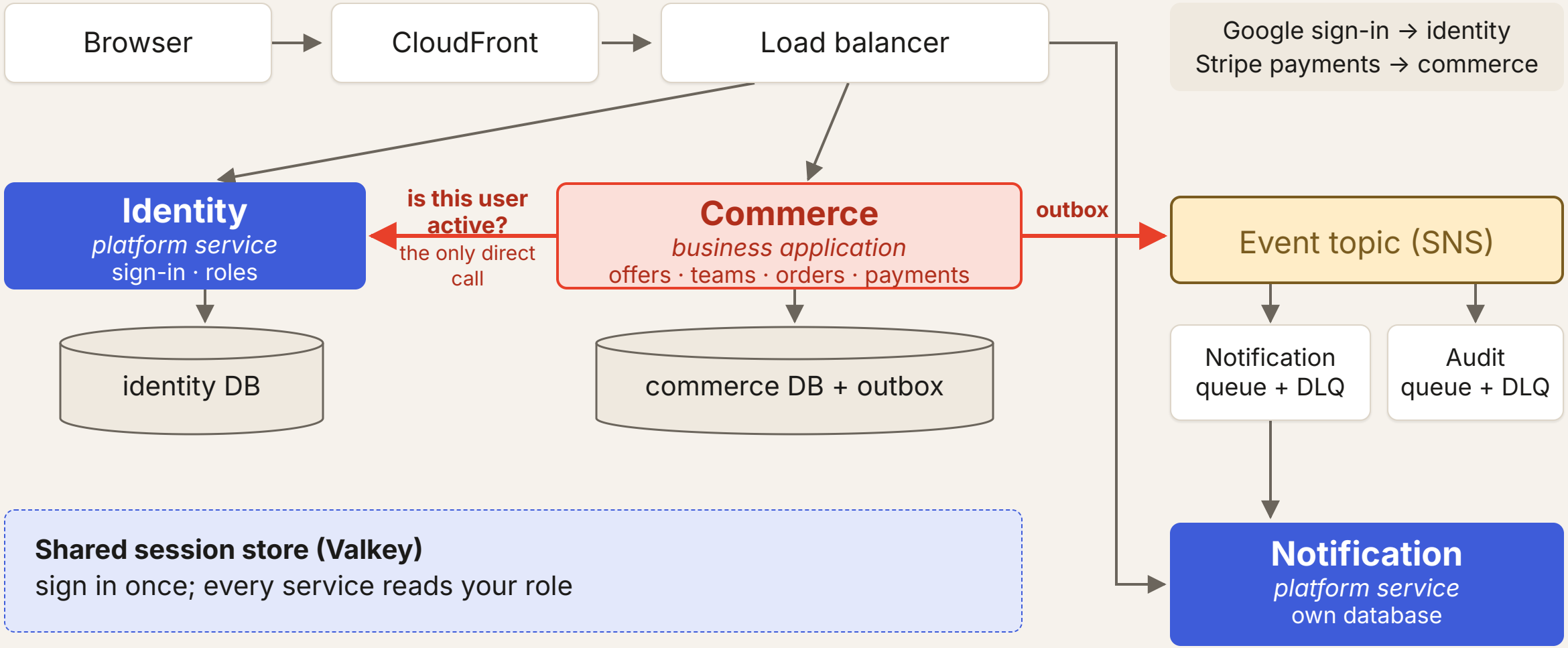
# Scale-out and resilience results

| Check | Target | Result |
|---|---|---|
| V3 HTTP auto-scale | ≥ 80% of linear from 1 → 4 tasks; p95 < 800 ms outside 3 min after scale-out | [FILL 19 Oct] |
| V4 queue scaling | 2,000 TeamExpired → 4 tasks, drain, DLQ 0 | [FILL 19 Oct] local IT: 141 s end to end |
| V5 self-heal | stop 1 of 2 tasks, 0 non-2xx, replacement < 2 min | [FILL 19 Oct] |
| V7 poison message | 5 receives → DLQ → alarm | [FILL 19 Oct] local: verified 15 Oct |
| Identity outage | browse 200; checkout 500 → redelivered once | [FILL 19 Oct] local: verified |
| Distributed up + smoke | 3 services stable, 9 smoke checks | 17 Oct ×2: tasks 2/2 · 1/1 · 1/1; 9/9 checks; SNS→SQS verified with synthetic events. Buyer E2E (#460): [FILL 19 Oct] |
| Teardown | all resources destroyed, absence check clean | 115 resources, 709 s (17 Oct); 113, 784 s (16 Oct) |

*Anything not run on AWS stays "not run on AWS; local result: ...". Never upgrade a local result to an AWS one.*

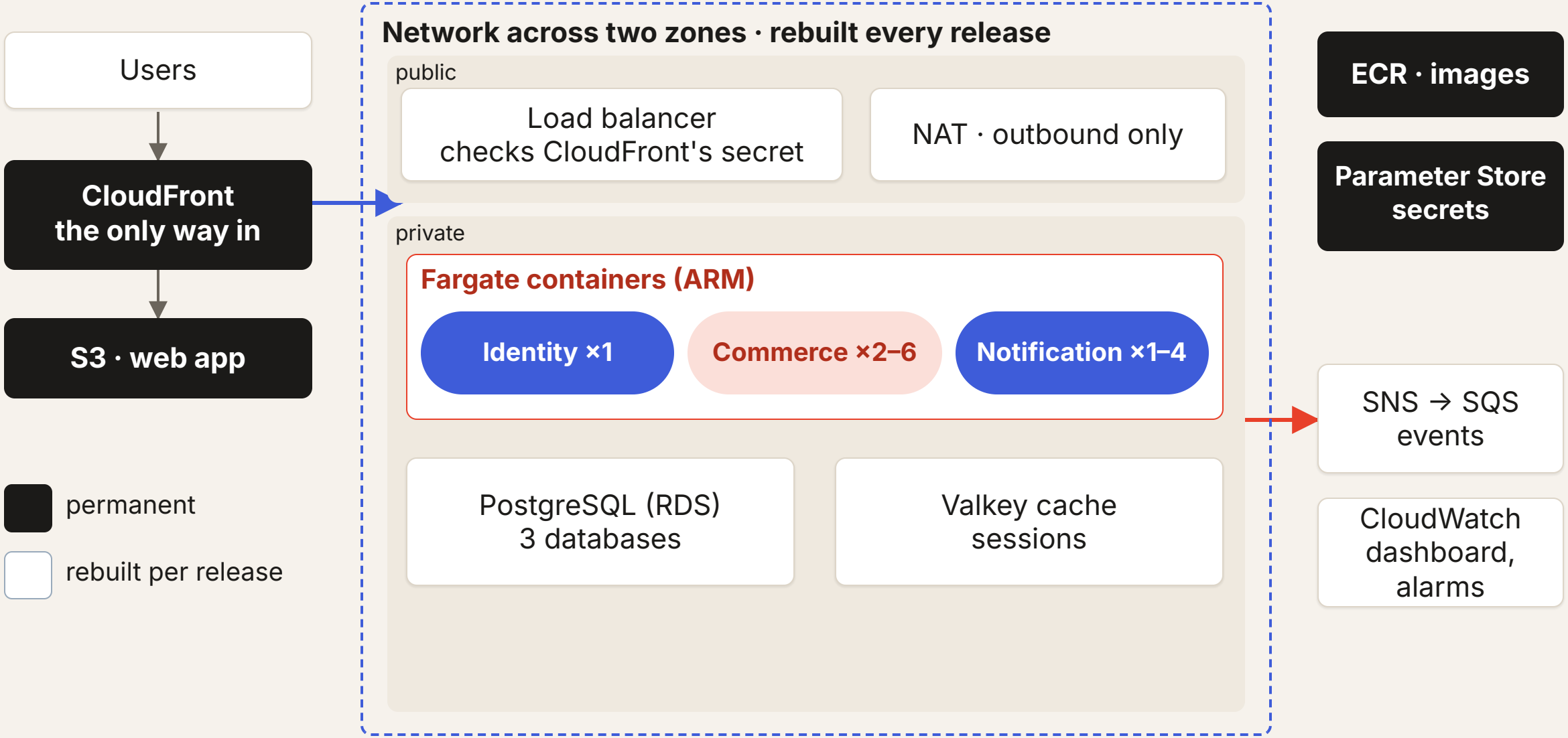
WZ Eric

# Logical view, simplified



LJ Liu Jin

# Physical view, simplified



YQ Yorik

# Team expiry, numbered steps (fallback for slide 16)

